

10-Day Streamlit Learning Plan

A practical beginner-to-project roadmap for building interactive Python, data, and AI applications.

Goal: By the end of 10 days, you should be able to build a polished Streamlit application, connect it to data or AI services, and prepare it for deployment.

Day 1 – Introduction to Streamlit

Focus: Learn what Streamlit is, why it is useful for data and AI applications, installation, and the basic application structure.

Topics to Learn
• Install Python and Streamlit
• Create your first app with <code>st.title()</code> and <code>st.write()</code>
• Run an app using <code>streamlit run app.py</code>
• Understand Streamlit's rerun model

Mini Project: Build a simple personal introduction page.

Day 2 – Text, Markdown and Basic UI

Focus: Learn how to create attractive content using headings, text, Markdown, code blocks, and dividers.

Topics to Learn
• <code>st.title()</code> , <code>st.header()</code> , <code>st.subheader()</code>
• <code>st.write()</code> and <code>st.markdown()</code>
• <code>st.code()</code> and <code>st.divider()</code>
• Organize content into logical sections

Mini Project: Create a Python learning notes application.

Day 3 – User Input Widgets

Focus: Learn how to collect information from users using Streamlit widgets.

Topics to Learn
• <code>st.text_input()</code>
• <code>st.number_input()</code> and <code>st.slider()</code>
• <code>st.selectbox()</code> and <code>st.multiselect()</code>
• <code>st.checkbox()</code> , <code>st.radio()</code> , and <code>st.button()</code>

Mini Project: Build a student grade calculator.

Day 4 – Layouts and UI Design

Focus: Learn how to structure applications using columns, containers, tabs, expanders, and sidebars.

Topics to Learn
• st.sidebar
• st.columns()
• st.container()
• st.tabs() and st.expander()
• Create clean dashboards

Mini Project: Create a multi-section student dashboard.

Day 5 – Session State and Interactivity

Focus: Understand how Streamlit manages application state and how to preserve values between reruns.

Topics to Learn
• Understand st.session_state
• Store counters and user selections
• Create interactive workflows
• Use callbacks where appropriate

Mini Project: Build an interactive task or counter application.

Day 6 – Data Handling and Tables

Focus: Learn how to load, display, filter, and manipulate data inside Streamlit.

Topics to Learn
• Read CSV files with pandas
• Display data with st.dataframe()
• Use st.table()
• Add filters and search controls

Mini Project: Build a simple employee or sales data explorer.

Day 7 – Charts and Visualizations

Focus: Learn how to turn data into useful visual dashboards.

Topics to Learn
• <code>st.line_chart()</code>
• <code>st.bar_chart()</code> and <code>st.area_chart()</code>
• Use pandas and Matplotlib/Plotly when needed
• Create KPI metrics with <code>st.metric()</code>

Mini Project: Build a sales performance dashboard.

Day 8 – Forms, Files and Advanced Components

Focus: Learn practical features needed for real-world applications.

Topics to Learn
• <code>st.form()</code> and <code>form_submit_button()</code>
• File upload with <code>st.file_uploader()</code>
• Download files with <code>st.download_button()</code>
• Progress indicators and status messages

Mini Project: Create a CSV upload and analysis application.

Day 9 – Streamlit for AI Applications

Focus: Use Streamlit as a frontend for AI and LLM applications.

Topics to Learn
• Connect Streamlit to Python AI code
• Create chatbot interfaces
• Handle API keys securely
• Display AI responses and conversation history

Mini Project: Build a basic AI chatbot interface.

Day 10 – Project, Testing and Deployment

Focus: Combine the concepts into a complete application and learn the basics of deployment.

Topics to Learn
• Create a project structure
• Improve UI and error handling
• Test different user inputs
• Prepare requirements.txt
• Deploy using Streamlit Community Cloud or another hosting platform

Mini Project: Build and deploy a complete AI/data dashboard.

Suggested Final Project

AI-Powered Data Dashboard – Create a Streamlit application where users upload a CSV file, view KPIs and charts, filter the data, and optionally ask an AI assistant questions about the dataset.

Recommended stack: Python + Streamlit + Pandas + Plotly + an LLM API + GitHub

Daily practice: Spend 60–90 minutes learning concepts and 60–90 minutes implementing the day's mini project.

Outcome: A portfolio-ready Streamlit project that demonstrates Python, data visualization, UI development, and AI integration.